

# **THE PROMPT LADDER**

**Name: Jiya Yadav**

**Track: FlyRank AI Internship · Backend**

**Week: 2 Assignment A3**

## **What This Document Is**

A prompt ladder built from real runs.

Each version adds exactly one change. The output is compared against the previous version. Notes describe what changed in the output — not just what changed in the prompt.

The point is not a perfect result. The point is knowing which ingredient earned its place.

## **Baseline (V0) — Weak Prompt**

### **Prompt:**

“Write backend code for a todo API”

### **Output (Representative Excerpt):**

```
# AI chose Node.js + Express + PostgreSQL + Prisma ORM
# Without being asked
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
model Todo {
  id        String  @id @default(uuid())
  title     String
  completed Boolean  @default(false)
  createdAt DateTime @default(now())
}
```

**Setup required:**

```
npm install express @prisma/client zod
```

```
npx prisma migrate dev --name init
```

**Notes**

What changed in the prompt:

Nothing. This is the starting point.

**What improved in the output:**

Nothing meaningful. It produced something runnable but immediately violated the assignment constraints — it added a PostgreSQL database, an ORM, and external dependencies without being asked for any of them.

**What still failed:**

- Chose Node.js and Express without being told to
- Added a full database (PostgreSQL + Prisma) — a major assumption
- Required environment variables and migrations just to run
- No status codes enforced
- No auto-increment — used UUID from client
- No validation logic
- Completely ignores the actual assignment constraints

**What I'd try next:**

Add a clearer goal. The prompt gives no definition of what "good" looks like — so the model decided everything itself.

**Version 1 — Clearer Goal**

Layer Added: Clearer Goal

**Prompt:**

“Write backend code for a todo API. It should be a complete REST CRUD API with proper HTTP status codes and validation.”

**Output (Representative Excerpt):**

```
// Still Node.js + Express + PostgreSQL + Prisma
```

```
// But now with explicit status codes and Zod validation
```

```
app.get('/api/todos', async (req, res, next) => {
  const todos = await prisma.todo.findMany({ orderBy: { createdAt: 'desc' } });
  res.status(200).json({ data: todos });
});
```

```
app.post('/api/todos', async (req, res, next) => {
  const validatedData = createTodoSchema.parse(req.body);
  const todo = await prisma.todo.create({ data: validatedData });
  res.status(201).json({ data: todo });
});
```

```
// Zod validation error handler
if (err.name === 'ZodError') {
  return res.status(400).json({
    error: 'Validation Error',
    details: err.errors.map(e => ({ field: e.path.join('.'), message: e.message })))
  });
}
```

## **Notes**

### **What changed in the prompt:**

Added a clear goal: complete REST CRUD with proper HTTP status codes and validation.

### **What improved in the output:**

Status codes became explicit and intentional — 200, 201, 204, 400, 404 all appeared. Zod validation was introduced with proper error messages. The output moved from a rough sketch to something structurally closer to a real API.

### **What still failed:**

- Still used PostgreSQL and Prisma — database assumption unchanged
- Client still provided UUID — no server-side ID generation
- No in-memory constraint respected — model still decided storage itself

- Zod errors mapped to 400, not 422

### **What I'd try next:**

Add real context — the model still doesn't know this is an internship assignment with specific constraints.

## **Version 2 — Real Context**

Layer Added: Real Context

### **Prompt:**

Write backend code for a todo API. It should be a complete REST CRUD API with proper HTTP status codes and validation. This is for an internship assignment that requires in-memory storage only — no database.

### **Output (Representative Excerpt):**

```
// Switched to in-memory array — database dropped entirely
const todos = [];
const findTodoById = (id) => todos.find((t) => t.id === id);
app.post('/api/todos', (req, res, next) => {
  const validatedData = createTodoSchema.parse(req.body);
  const newTodo = {
    id: crypto.randomUUID(),
    title: validatedData.title,
    completed: false,
    createdAt: new Date().toISOString(),
  };
  todos.push(newTodo);
  res.status(201).json({ success: true, data: newTodo });
});
```

### **Notes**

#### **What changed in the prompt:**

Added real constraint: internship assignment, in-memory only, no database.

### **What improved in the output:**

The database disappeared entirely. No Prisma, no PostgreSQL, no migrations. The model used a plain JavaScript array with `crypto.randomUUID()`. The output became self-contained and runnable without environment setup. Complexity dropped, focus improved.

### **What still failed:**

- Output format undefined — code dumped without explanation
- No design reasoning included
- No curl examples for testing
- Still no constraint on which framework to use
- Still Node.js — not FastAPI

### **What I'd try next:**

Define an output format. The code exists but there is no explanation of decisions.

## **Version 3 — Specified Output Format**

Layer Added: Output Format

### **Prompt:**

Write backend code for a todo API. It should be a complete REST CRUD API with proper HTTP status codes and validation. This is for an internship assignment that requires in-memory storage only — no database.

Provide:

- The full FastAPI code
- A short explanation of design decisions
- Example curl requests

Output (Representative Excerpt):

```
// Switched to a Map for O(1) lookups
```

```
export const todoStore = new Map();
```

```
// Folder structure introduced
```

```
todo-api/
```

```
├── src/
```

```
|   ├── app.js
```

```
|   └── store.js
```

```
| |— schemas/todo.schema.js
| |— routes/todo.routes.js
```

// Routes now separated

```
router.get('/:id', (req, res, next) => {
  const todo = todoStore.get(req.params.id);
  if (!todo) {
    return res.status(404).json({ success: false, error: 'Todo not found' });
  }
  res.status(200).json({ success: true, data: todo });
});
```

### **Design decision excerpt included:**

"A standard list requires  $O(N)$  linear scan to look up or delete an item by ID. A Python dictionary yields  $O(1)$  constant time complexity for reads, updates, and deletes."

### **Notes:**

#### **What changed in the prompt:**

Required structured output in three named sections plus named the framework explicitly (FastAPI).

#### **What improved in the output:**

The output became organised and usable. Design decisions appeared with actual reasoning — not just what was built but why. Curl examples made it immediately testable. A stranger could pick it up and run it. The model also upgraded storage from an array to a Map with  $O(1)$  lookup reasoning.

#### **What still failed:**

- Model still used Node.js despite the prompt saying FastAPI
- Design reasoning was present but surface-level
- No explicit quality bar — model decided what "correct" meant
- No auto-increment IDs — still using UUID

#### **What I'd try next:**

Add explicit quality criteria so the model knows what the API must actually satisfy.

## **Version 4 — Quality Criteria**

Layer Added: Quality Criteria

### **Prompt addition:**

“The API must:

- Auto-increment IDs (client must not provide ID)
- Allow partial updates via PUT
- Return 404 for unknown IDs
- Return 422 for validation errors — not 400
- Use response\_model on all endpoints”

### **Output (Representative Excerpt):**

```
# Finally switched to FastAPI
```

```
# Auto-increment via itertools.count
```

```
id_counter = itertools.count(start=1)
```

```
todo_db: dict[int, "TodoResponse"] = {}
```

```
class TodoUpdate(BaseModel):
```

```
    title: Optional[str] = Field(default=None, min_length=1)
```

```
    description: Optional[str] = Field(default=None)
```

```
    completed: Optional[bool] = Field(default=None)
```

```
@app.put("/todos/{todo_id}", response_model=TodoResponse)
```

```
def update_todo(todo_id: int, payload: TodoUpdate):
```

```
    existing = todo_db[todo_id]
```

```
    update_data = payload.model_dump(exclude_unset=True)
```

```
    updated = existing.model_copy(update=update_data)
```

```
    todo_db[todo_id] = updated
```

```
    return updated
```

### **Notes**

#### **What changed in the prompt:**

Added five explicit correctness requirements with no room for interpretation.

#### **What improved in the output:**

The model finally switched to FastAPI. IDs became server-generated via `itertools.count`. Partial PUT worked correctly using `exclude_unset=True`. FastAPI's native Pydantic integration returned 422 automatically — no custom handler needed. Every endpoint had `response_model`. The output aligned with professional API standards.

### **What still failed:**

The model produced the correct structure but never verified it had actually met each requirement. It could silently miss something and not flag it. No self-check existed.

### **What I'd try next:**

Add a verification requirement — force the model to review its own output before finishing.

## **Version 5 — Verification Requirement**

Layer Added: Verification Requirement

### **Prompt addition:**

After generating the code, review it and explicitly confirm that each quality criterion is satisfied. If any are missing, fix them before finalizing.

### **Output (Verification Section Excerpt):**

Quality Criterion Review & Explicit Confirmation:

#### **☑ In-memory storage only**

CONFIRMED. Data kept in `todo_db: dict[int, TodoResponse]` — no external database.

#### **☑ Auto-increment IDs (client must not provide ID)**

CONFIRMED. `TodoCreate` excludes `id` field.

IDs generated via `next(id_counter)`.

#### **☑ Allow partial updates via PUT**

CONFIRMED. `TodoUpdate` uses optional fields and `exclude_unset=True`.

#### **☑ Return 404 for unknown IDs**



CONFIRMED. HTTPException(404) raised across GET, PUT, DELETE when todo\_id missing from todo\_db.

✓ Return 422 for validation errors (not 400)

CONFIRMED. FastAPI Pydantic integration returns 422 automatically.

✓ response\_model on all endpoints

CONFIRMED.

POST → response\_model=TodoResponse

GET → response\_model=List[TodoResponse]

GET → response\_model=TodoResponse

PUT → response\_model=TodoResponse

DELETE → response\_model=None (204 No Content)

---

## **Notes**

### **What changed in the prompt:**

Required the model to self-review against stated criteria and fix anything missing before finalising.

### **What improved in the output:**

The model produced an explicit checklist confirming every requirement. This was the first version where I didn't have to manually check whether status codes were correct or whether response\_model was on every endpoint. The verification section caught that DELETE needed response\_model=None explicitly — it was added and confirmed.

### **What still failed:**

Self-review is only as good as model consistency. It cannot guarantee runtime correctness. The model confirmed all requirements were met — but that confirmation cannot be tested without running the code. This layer reduces silent errors. It does not eliminate them.

### **What I'd try next:**

Add failure cases or force the model to generate a test that would break the API if a requirement was missing.

## Final Reusable Prompt

Cleaned up. Ready for someone else to use without me in the room.

Write a complete REST CRUD Todo API using FastAPI.

Context:

This is for an internship assignment that requires in-memory storage only — no database.

Requirements:

- Auto-increment IDs (client must not provide ID)
- Proper HTTP status codes:
  - 200 read
  - 201 create
  - 204 delete
  - 404 unknown ID
  - 422 validation errors
- Pydantic validation on all inputs
- Partial updates allowed via PUT
- response\_model used on all endpoints
- No external dependencies beyond FastAPI and Pydantic

Output format:

1. Full working FastAPI code
2. Short explanation of design decisions
3. Example curl commands for each endpoint

After generating, review the output against each requirement above and explicitly confirm it is satisfied. Fix anything missing before finalizing.

## **Before / After — One Honest Example**

✗ What I started with:

“Write backend code for a todo API.”

**What it produced:**

A PostgreSQL + Prisma + Node.js API that required database migrations to run. Nothing about the actual assignment constraints.

### ✓ **What I finished with:**

Write a complete REST CRUD Todo API using FastAPI with auto-increment IDs, strict HTTP status codes, Pydantic validation, partial updates via PUT, and in-memory storage only. Structure output as full code, design decisions, and curl examples. After generating, verify each requirement is met and fix anything missing.

### **What it produced:**

A self-contained FastAPI service with verified correctness, explicit status codes, partial updates, and a checklist confirmation.

One is a guess dressed as a prompt.

One is a contract with the model.

### **What This Ladder Actually Taught**

- Clearer goal → Model chose better tools. Status codes appeared without being demanded individually.
- Real context → Model dropped the database entirely. Complexity reduced to what was needed.
- Output format → Output became usable by a stranger. Design reasoning appeared.
- Quality criteria → Model stopped deciding what "good enough" meant. Specific requirements produced specific behaviour.
- Verification → Model caught its own gaps. DELETE response\_model=None was added after self-review.

### **One honest failure moment:**

Version 3 did not work as expected.

The prompt named FastAPI explicitly. The model still used Node.js. Naming a framework in the output format section was not enough. It took explicit quality criteria in Version 4 to force the switch.

Lesson: Naming a tool in a format instruction is not the same as requiring it. Requirements enforce. Format instructs.